

4.5.4. Procesamiento en línea (pipeline)

Una forma más para acelerar la máquina consiste en conformar el hardware con varias unidades funcionales y después unir las para que trabajen en línea o en forma escalonada (pipeline). * En la figura 2-5 se observó cómo funciona una computadora con cinco unidades escalonadas. En la figura 4-24 se muestra que si la instrucción 1 se extrae en el ciclo 1, se ejecutará en el ciclo 5. En forma parecida, si la instrucción 4 se extrae en el ciclo 8, se decodifica en el 9, etc., se ejecutará en el ciclo 12. Bajo condiciones óptimas, en cada ciclo de ahí en adelante se ejecuta una instrucción, dando un promedio de ejecución de una instrucción por ciclo, en lugar de una cada cinco ciclos.

Desafortunadamente, hay estudios que han demostrado que alrededor del 30% de las instrucciones son saltos y éstos hacen estragos en la línea de procesamiento. Los saltos pueden clasificarse en tres categorías: incondicionales, condicionales e iterativos. Un salto incondicional indica a la computadora que detenga la extracción de instrucciones en forma consecutiva y se dirija a una dirección específica. Un salto condicional verifica una condición y salta si ésta se cumple. Un ejemplo típico es una instrucción que chequea un registro y salta si éste contiene un valor de cero. En caso contrario, no se lleva a cabo el salto y el control continúa en la secuencia actual.

Por lo general las instrucciones iterativas decrementan un contador de iteraciones y saltan al inicio si el contador no es cero (v.g. todavía hay iteraciones por hacer). Estas instrucciones son un caso especial importante de los saltos condicionales, ya que se sabe de antemano, que casi siempre habrá ese salto.

Considérese lo que sucede en la línea de procesamiento (pipeline) de la figura 4-24 cuando se encuentra una instrucción (señalada como B) de salto condicional. La siguiente instrucción a ejecutar debería ser la siguiente a la del salto,

*Nota del R.T.: Cada unidad trabaja en una tarea específica como en una línea de producción industrial.

VARIAS
UNIDADES FUNCIONALES

Ciclo													
	1	2	3	4	5	6	7	8	9	10	11		
Extracción de instrucciones	1	2	B					4	5	6	7	8	9
Decodificación de instrucciones		1	2	B					4	5	6	7	8
Cálculo de direcciones			1	2	B					4	5	6	7
Extracción de operandos				1	2	B					4	5	6
Ejecución					1	2	B					4	5

Fig. 4-24. Unidad de procesamiento en línea de 5 etapas. Las instrucciones están etiquetadas con números. La instrucción marcada como B es un salto condicional.

pero también podría ser la dirección a la cual saltó, llamada **blanco del salto**. En virtud de que el extractor de instrucciones no sabe cual sigue sino hasta que se ejecuta el salto, se demora y no puede continuar hasta la ejecución de éste. En consecuencia la línea se vacía. Sólo después de que termina el ciclo 7, se sabe qué instrucción sigue.

A la pérdida de cuatro ciclos causada de hecho por el salto se le denomina **penalización por salto**. Resulta evidente que con un salto en una de cada tres instrucciones, la disminución en el desempeño es sustancial.

Se han realizado una gran cantidad de investigaciones alrededor del problema de recuperar algo de este desempeño (DeRosa y Levy, 1987; McFarling y Hennessy, 1986; Hwu y col. 1989 y, Lilja, 1988). Lo más sencillo consiste en esperar que el salto no se realice y continuar procesando como si el salto fuera una simple instrucción aritmética. Si resulta que en efecto, el salto no se realiza, no se ha perdido nada. Si por el contrario el salto se efectúa, se deben eliminar las instrucciones que estén actualmente en la línea y empezar de nuevo.

Esta opción presenta sus propios problemas. En algunas máquinas, se puede modificar un registro como el resultado intermedio de un cálculo de direcciones. Si una instrucción eliminada modificó a uno o más registros, estos deben ser restaurados, lo que significa que debe haber un mecanismo para grabar los valores originales. Resulta instructivo hacer un modelo de la pérdida en el desempeño.

Sea P_j la probabilidad de que una instrucción sea un salto.

Sea P_r la probabilidad de que el salto se realice.

Sea b la penalización por salto.

El tiempo promedio de ejecución (en ciclos) se puede calcular como la suma ponderada de los dos casos: instrucciones regulares e instrucciones de salto.

$$\text{Tiempo promedio de ejecución} = (1 - P_j)(1) + P_j[P_r(1 + b) + (1 - P_r)(1)]$$

Un poco de álgebra revela que el tiempo promedio por instrucción es $1 + bP_jP_r$. Luego la eficiencia de ejecución es $1/(1 + bP_jP_r)$. Asignando los valores (típicos estimados) de $b = 4$, $P_j = 0.3$ y $P_r = 0.65$, la máquina corre a menos del 60% de su capacidad potencial.

¿Qué puede hacerse para mejorar el desempeño? Para empezar, si se pudiera predecir la dirección del salto, se podría extraer la instrucción apropiada (siguiente) y eliminar la penalización. En la fórmula para la eficiencia se puede entonces reemplazar P_r por la probabilidad P_w de que la suposición sea falsa.

Existen dos clases de predicciones: estáticas (al tiempo de compilación) y dinámicas (al tiempo de ejecución). Con predicción estática, el compilador hace

una suposición para cada una de las instrucciones de salto que genera. Suponer por ejemplo, en las instrucciones iterativas que el salto regresará al inicio de la iteración, es correcta la mayoría de las veces. Cuando se trata de una condición improbable, tal como una llamada al sistema que genera un código de error, es muy posible que el salto no se lleve a cabo. En muchos casos se utilizan diferentes instrucciones para estos casos y el solo hecho de observar el código de operación proporciona una buena sugerencia.

Otro esquema más elaborado para los diseñadores de máquinas consiste en proporcionar dos códigos de operación para cada tipo de salto y hacer que el compilador use el primero si cree que el salto se llevará a cabo y el segundo en caso contrario. En forma alternativa, para programas de uso intensivo, se pueden correr primero en un simulador, registrando, el comportamiento de cada salto; luego, el programa binario puede "parcharse" con el código de operación apropiado (probablemente se realice, probablemente no se realice).

En el método de predicción dinámica, el microprograma construye, durante la ejecución, una tabla con las direcciones que contienen saltos y guarda un registro del comportamiento de cada uno. Este método tiende a hacer más lenta la máquina al llevar el registro, de modo que requiere de alguna ayuda del hardware. Algunas mediciones han mostrado que de esta manera no es difícil lograr un 90% de precisión.

La predicción de los saltos no es el único punto de ataque. Otra opción es determinar por anticipado en la línea, hacia que dirección será el salto. Algunas pruebas, como el salto si es *igual* son mucho más fáciles de hacer que en salto si *menor que*. Las primeras se pueden hacer con un comparador, mientras que las segundas requieren de un ciclo completo de trayectoria de datos para hacer una resta. Con este enfoque, siempre que el microprograma encuentre un salto, hace una verificación rápida en una etapa temprana de la línea para ver si puede resolver el salto de inmediato, en cuyo caso sabe en donde continuar con la extracción de la siguiente instrucción.

Los compiladores pueden ser de gran ayuda, por ejemplo, cuando el programador escribe una iteración variando i desde 1 hasta 10, el compilador puede verificar i para ver si es igual a 10, en vez de ver si es menor de 11, de manera que el microprograma puede librar la cuestión con una comparación en lugar de una resta.

Para ocuparse de aquellos saltos que no se pueden resolver en las primeras etapas, el compilador puede tratar de encontrar algo útil para que la computadora trabaje, mientras espera que se ejecute el salto. Considérese la declaración aritmética seguida de una comparación, de la figura 4-25(a). Un compilador con una optimización hábil pudiera producir un código más parecido al de la figura 4-25(b), el cual no está en lenguaje Pascal legal, pero muestra el orden de eventos. Primero genera el código para hacer la comparación, luego hace la operación aritmética. Después de que el salto ha ingresado en la línea de proce-


```

a := b + c;
if b < c
then declaración;
(a)

```

```

if b < c
a := b + c;
then declaración
(b)

```

Fig. 4-25. (a) Un fragmento en lenguaje Pascal. (b) Como pudiera ser el tratamiento por parte del compilador.

samiento, le siguen varias instrucciones ordinarias, las cuales sin lugar a duda deben ejecutarse, así que no hay necesidad de hacer ninguna predicción ni eliminación. Para usar esta técnica con efectividad, los microprogramadores y los diseñadores de compiladores tuvieron que trabajar en estrecha coordinación durante el diseño.

Si todo lo demás falla, siempre existe la posibilidad de seguir ambas trayectorias en paralelo. Para hacerlo, se requieren dos líneas de procesamiento en el hardware y no evita el problema de eliminación. No obstante, este método se utiliza en ocasiones en las computadoras de orden superior, en las que el aspecto importante es el desempeño y no el precio. Desde luego, si cada trayectoria se topa con otro salto antes de resolver el primero, el asunto se vuelve aún más complicado. Probablemente no sea la mejor idea tener "flojeando" algunas docenas de líneas de procesamiento, para enfrentar el peor de los casos.

Después de toda esta teoría respecto a lo que podría hacerse, regrese por un momento a la máquina de ejemplo. Esta, no tiene distintas unidades de hardware para extracción, decodificación, etc., de modo que no es posible formar estas líneas de procesamiento, pero con cambios menores al microprograma, se puede lograr un número limitado de traslapes entre las microinstrucciones de extracción y ejecución, lo que también es una forma de realizar las líneas.

En la figura 4-10, por ejemplo, si la siguiente microinstrucción pudiera extraerse de alguna manera durante el subciclo 4, ya no se necesitaría el subciclo 1 y el reloj sólo generaría impulsos para los subciclos 234234234... El principal problema surge con las microinstrucciones de salto condicional. Si la máquina esperara a que las líneas de estado de la ALU estén disponibles antes de iniciar la extracción de la microinstrucción siguiente, sería demasiado tarde. El ciclo habría prácticamente terminado y se podría conseguir poco traslape. En algunas máquinas el problema se resuelve usando las líneas de estado de la ALU del ciclo anterior que, por supuesto, deberán guardarse en biestables para evitar que se desvanezcan. Estos valores están disponibles al comienzo de cada microinstrucción y por tanto puede empezar la siguiente extracción inmediatamente, mucho antes de que se termine la ejecución de la microinstrucción en curso. No es necesario decir que esto complica en gran medida la vida del microprogramador.

Desde el punto de vista técnico, es posible escribir microprogramas para una máquina de este estilo en una forma razonable. Por ejemplo, para examinar alguna palabra de la memoria de anotaciones y saltar si es negativa, el microprograma puede pasar la palabra por la ALU en una microinstrucción que *no* contenga un salto. La siguiente microinstrucción será una no-operación con un salto condicional. Es decir, no hará nada excepto examinar los bits de estado de la ALU, que se habían memorizado, y saltar. El precio a pagar por programar en forma razonable es doblar la duración del salto, lo que no es muy deseable.

La única solución es conseguirse un frasco lleno de aspirinas y tratar de sacar el máximo provecho de cada situación. Por ejemplo, el microprogramador podría hacer una hipótesis de dónde suele saltar la microinstrucción y empezar a hacer el trabajo que probablemente se necesite. Por desgracia, si el salto fuera a otra alternativa, sería necesario eliminar parte de lo hecho.

He aquí un ejemplo sencillo: reescribimos las líneas 11 a 18 de la figura 4-16 para una máquina con extracción y ejecución traslapadas de microinstrucciones. Los resultados se muestran en la figura 4-26(a). En este ejemplo tenemos suerte, ya que después de ejecutar la línea 11 sabemos que seguirá el código de ADDD o de SUBD, que empiezan por la misma microinstrucción. Por tanto, basta poner la microinstrucción común en la línea 12 para ocupar la máquina mientras se realiza el salto.

Un ejemplo menos elegante es el de la figura 4-26(b). En la línea 52 sabemos que vamos a entrar en la microrrutina de PSHI o en la de POPI, pero, lamentablemente no empiezan por la misma microinstrucción. Supongamos, sin embargo, que un análisis estadístico mostró que los programas del Mac-1 usan PSHI muchas más veces que POPI. Entonces podríamos proceder como se indica en la figura 4-24(b). Para PSHI todo va bien, pero para POPI, cuando se llega a la línea 56, se ha iniciado una lectura en memoria de una palabra incorrecta. Según los detalles específicos del hardware, puede ser posible o no abortar una lectura de memoria a medio realizar sin perturbar el funcionamiento.

```

11: alu := tir;
12: mar := ir; rd; if n then goto 16;
13: rd;
14: ac := mbr + ac; goto 0;
16: ac := ac + 1; rd;
17: a := inv (mbr);
18: ac := ac + a; goto 0;
    
```

(a)

```

52: alu := tir;
53: mar := ac; rd; if n then goto 56;
54: sp := sp + (-1); rd;
55: mar := sp; wr; goto 10;
56: rd;
57: mar := sp; sp := sp + 1; rd;
58: rd;
59: mar := ac; wr; goto 10;
    
```

(b)

Fig. 4-26. Dos ejemplos con extracción y ejecución traslapada de microinstrucciones

to. Si no podemos abortarla, simplemente la terminamos y luego iniciamos la lectura correcta. En este ejemplo POPI requiere 15 microinstrucciones en lugar de las 13 de la figura 4-16. Sin embargo, si el uso del traslape reduce el tiempo básico de microinstrucción en 15%, incluso POPI será más rápida que antes.

La situación se volvería en extremo compleja si se utilizaran saltos condicionales de alternativas múltiples, formados haciendo el O lógico de NZDA con los bits menos significativos de ADDR en vez de comprobaciones sobre los bits individuales. Además, en muchas máquinas, el traslape de la extracción y la ejecución de las microinstrucciones se hace de tal modo que, al principio de la ejecución de la microinstrucción n , la elección de la microinstrucción $n+1$ ha de hacerse utilizando los bits de estado de la ALU de la microinstrucción $n-1$. No vamos a continuar profundizando en el tema, pero con esto basta para entender completamente la definición de microprogramación dada por Rosin.

4.5.5. Memoria caché

Por tradición, las CPU han sido siempre más rápidas que las memorias y aunque se han desarrollado las segundas, los CPU también lo han hecho, manteniendo así el desequilibrio. Lo que esto significa en la práctica es que después de que la CPU envía una solicitud a la memoria, debe permanecer ociosa durante un tiempo sustancial, aguardando a que la memoria responda. Como se ha visto, es común que la CPU establezca una lectura de memoria durante un ciclo del bus y no obtenga los datos hasta dos o tres ciclos más tarde, aun cuando no haya estados de espera.

En realidad, el problema no es de tecnología sino económico. Los ingenieros saben como construir memorias tan rápidas como las CPU, pero son tan caras que equipar a una computadora con un megabyte o más es impensable (excepto quizá para las supercomputadoras, en las que prácticamente no hay límite y el precio no representa objeción). De ahí que la elección se reduce a tener una pequeña cantidad de memoria o una gran cantidad de memoria lenta. Lo preferible sería una gran cantidad de memoria a un bajo precio.

De sumo interés son las técnicas conocidas para combinar estos tipos de memoria a fin de obtener, a bajo precio, la velocidad de la memoria rápida y la capacidad de la memoria lenta. A la memoria pequeña y rápida se le llama caché (del francés *cacher* que significa esconder) y está bajo el control del microprograma. En seguida, se describirá cómo se utilizan y funcionan estas memorias. Se puede encontrar más información en (Agarwall op. cit., 1989; Farrens y Pleszkun, 1989; Kabakibo op. cit., 1987; Kessler op. cit., 1989; Pohm y Agarwall, 1983; Przybylski op. cit., 1989; Smith, 1982; y Wang op. cit., 1989).

Desde hace años es sabido que los programadores no accesan las memorias en forma completamente aleatoria. Si una referencia a memoria dada, es a la dirección A , es común que la siguiente referencia se realice en los alrededores de